

# Medical Image Diagnosis Assistant

**Domain:** Healthcare - Computer Vision

## Project Overview

Develop an AI-powered diagnostic assistant that analyzes medical images such as chest X-rays or retinal scans to detect abnormalities indicative of diseases. The system should leverage deep learning techniques to identify patterns and anomalies, providing decision support to medical professionals by highlighting potential areas of concern in the images.

## Business Context

Early and accurate diagnosis of medical conditions through imaging is critical for effective treatment and patient outcomes. Automating the detection of abnormalities can reduce diagnostic errors, speed up clinical workflows, and extend expert-level diagnostic capabilities to under-resourced healthcare settings. This project aims to create a reliable, interpretable AI tool that assists radiologists and ophthalmologists in screening and diagnosis, ultimately improving healthcare delivery and patient care.

## Technical Objectives

- Preprocess and normalize medical image datasets (ChestX-ray14 or Diabetic Retinopathy Dataset) to ensure quality and consistency.
- Design and implement convolutional neural network architectures suitable for medical image classification and localization tasks.
- Incorporate domain-specific data augmentation techniques to improve model robustness.
- Utilize MONAI framework to facilitate medical imaging workflows and leverage PyTorch for model development.
- Implement explainability methods (e.g., Grad-CAM, saliency maps) to visualize model decision rationale.
- Develop a pipeline for training, validation, and testing with appropriate data splits and cross-validation strategies.
- Optimize model performance balancing sensitivity and specificity to minimize false negatives and false positives.
- Document the model development process, assumptions, and limitations clearly.

## System Architecture Guidelines

The system should follow a modular architecture comprising data ingestion and preprocessing modules, a deep learning model training and inference engine, and an explainability component for

visualizing model outputs. Data pipelines must handle large-scale image datasets efficiently with batch processing and caching. The model component should support GPU acceleration and allow easy experimentation with different architectures and hyperparameters. Outputs should include classification labels and localization heatmaps. The architecture must facilitate reproducibility and scalability, with clear separation between data handling, model logic, and evaluation components.

## Expected Deliverables

- A fully functional deep learning model capable of detecting abnormalities in chest X-rays or retinal images.
- Preprocessing scripts and data pipeline implementations.
- Training and evaluation code with configurable parameters.
- Visualizations demonstrating model explainability on sample images.
- A comprehensive technical report detailing methodology, experiments, results, and insights.
- Documentation of system architecture and usage instructions.

## Modeling Requirements

- Use convolutional neural networks tailored for medical image analysis, such as ResNet, DenseNet, or custom architectures.
- Incorporate transfer learning from pretrained models where appropriate.
- Apply data augmentation techniques specific to medical images (e.g., rotation, scaling, contrast adjustment).
- Implement multi-label classification if dataset contains multiple abnormality types.
- Integrate explainability techniques to provide interpretable model outputs.
- Ensure model generalization through validation on unseen data subsets.
- Address class imbalance using techniques such as weighted loss functions or oversampling.

## Evaluation Metrics

- Area Under the Receiver Operating Characteristic Curve (AUC-ROC) for classification performance.
- Accuracy, Precision, Recall (Sensitivity), and F1-score to assess detection quality.
- Specificity to evaluate false positive rates.
- Localization accuracy if bounding boxes or segmentation masks are available.
- Confusion matrix analysis to understand error distribution.
- Qualitative assessment of explainability visualizations by domain experts (if possible).

## Implementation Constraints

- Use PyTorch as the primary deep learning framework.
- Leverage MONAI for medical imaging-specific utilities and workflows.
- Ensure code compatibility with GPU acceleration.

- Maintain reproducibility with fixed random seeds and documented environment configurations.
- Adhere to ethical considerations regarding medical data privacy and responsible AI use.
- Limit model size and inference time to enable potential deployment in clinical settings.

## Stretch Goals

- Implement segmentation models to delineate abnormal regions precisely.
- Incorporate multi-modal data inputs if available (e.g., patient metadata alongside images).
- Develop a user interface prototype for interactive visualization of model predictions.
- Explore semi-supervised or self-supervised learning to leverage unlabeled data.
- Perform external validation on independent datasets to assess model robustness.
- Integrate uncertainty quantification to inform confidence in predictions.

## Skills Evaluated

- Deep learning model design and implementation for computer vision.
- Medical image preprocessing and augmentation techniques.
- Use of specialized frameworks such as MONAI and PyTorch.
- Model evaluation using appropriate metrics for imbalanced medical datasets.
- Explainability and interpretability of AI models.
- System design for modular, scalable AI pipelines.
- Documentation and technical reporting skills.
- Understanding of ethical considerations in AI for healthcare.

## Project Submission Format

Please follow the instructions below to ensure your project is submitted correctly and meets all requirements. Submissions not adhering to this format may not be accepted.

### 1. GitHub Repository Requirements

- Create a Public GitHub Repository.
- Name the repository appropriately for your project.
- Include a README.md with:
  - Project title and description
  - Setup and usage instructions
  - Dependencies or prerequisites
  - Brief explanation of your solution approach
- Regular Commits
- Commit your work frequently as you progress.
- Each commit should represent a logical, completed change.

- Write clear, descriptive commit messages.
- Avoid large, monolithic commits.
- Push to GitHub
- Ensure all required files are present and up to date.
- Do not include unnecessary files.
- Use a .gitignore file as needed.

## **2. Video Demonstration (Unlisted YouTube Video)**

- Create a short video (3–5 minutes).
- Explain your approach.
- Demonstrate your project running.
- Optionally walk through important code sections.
- Upload the video as Unlisted.
- Copy and submit the unlisted YouTube link.

## **3. Submission Checklist**

- GitHub repository with regular, meaningful commits.
- Complete README.md.
- All source code pushed to GitHub.
- Unlisted YouTube video link.

## **4. Submission Instructions**

- GitHub repository URL
- Colab Notebook link
- Unlisted YouTube video link

## **Best Practices & Tips**

- Commit often and write clear commit messages.
- Keep your repository organized and clean.
- Ensure your code runs without errors.
- Use branches for significant features.
- Double-check video visibility before submission.
- Example Commit Message: `git commit -m "Add input validation for registration form"`
- How to Set YouTube Video as Unlisted: When uploading, under "Visibility," select "Unlisted." Save and copy the video URL for submission.
- By following this format, you ensure your submission is professional, easy to review, and meets all technical requirements.